# 과적합 피하기

국립금오공과대학교
시종욱

# 목차

- 과적합 (과대적합, 과소적합)
- 데이터세트의 유형
- K-Fold Cross Validation
- 과적합 피하기 방법

# 과적합이란

- 과적합
  - ➢ 과소적합(Underfitting)
    - ✓ 모델의 용량이 작아 오차가 큰 현상

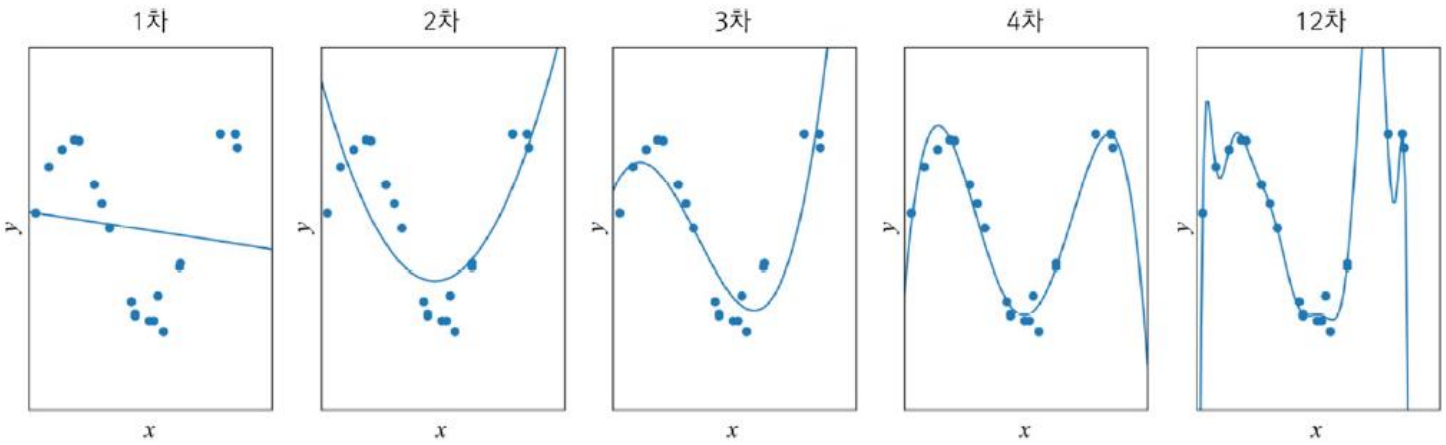



그림 1-13 과소적합과 과잉적합 현상

기계학습, 오일석 저, 한빛미디어

# 과적합이란

- 과적합
  - ➢ 과대적합(Overfitting)
    - ✓ $x_0$에서 빨간 막대 근방을 예측해야 하지만 빨간 점을 예측
    - ✓ 이유는 '용량이 크기' 때문이며 학습 과정에서 잡음까지 수용 → 과잉적합 현상
    - ✓ 적절한 용량의 모델을 선택하는 모델 선택 작업이 필요함

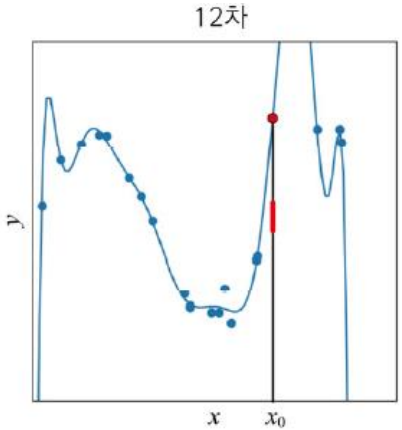
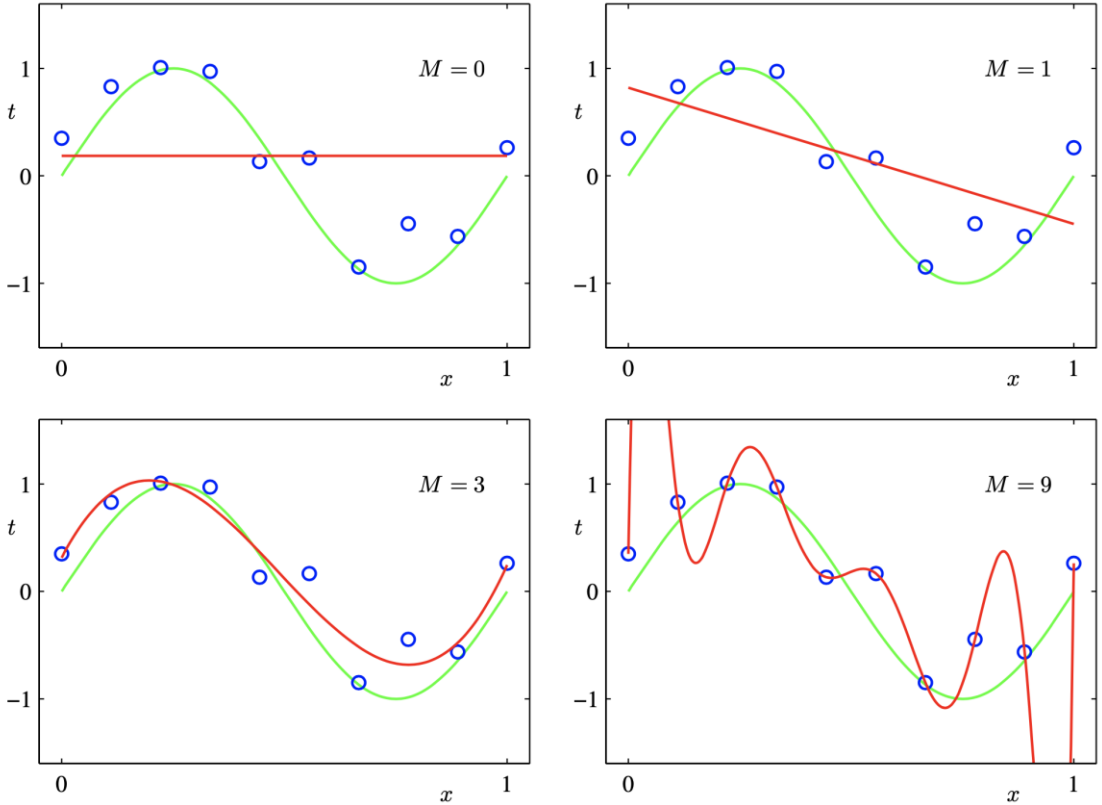



그림 1-14 과잉적합되었을 때 부정확한 예측 현상

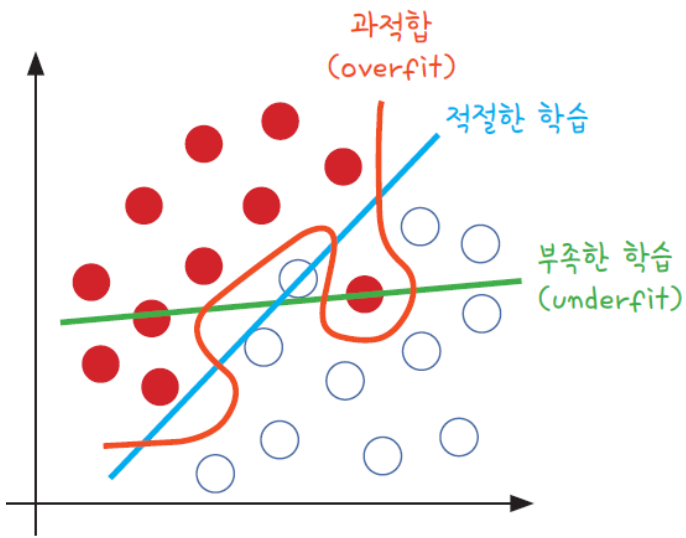
# 과적합이란




(Information science and statistics) Christopher M. Bishop - Pattern Recognition and Machine Learning-Springer (2006)

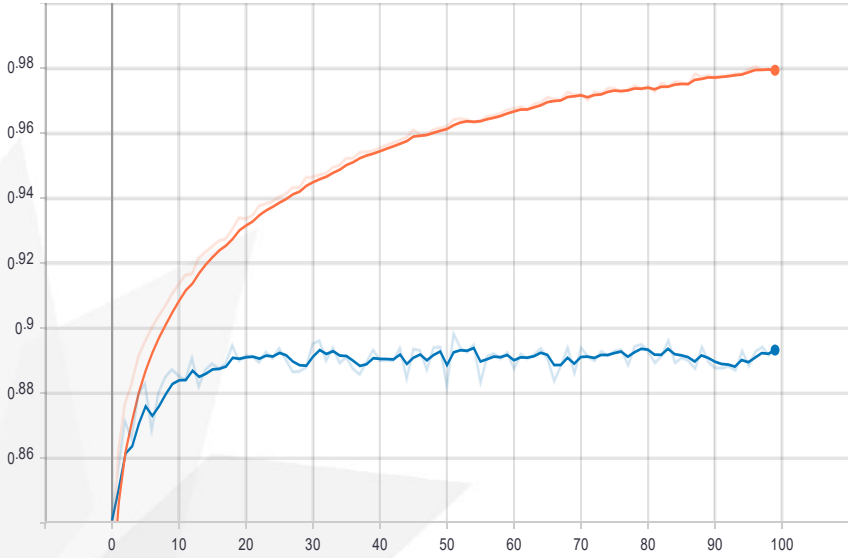
# 과적합

- 과대적합
  - ➢ 학습 데이터세트에서는 일정 수준 이상의 예측 정확도를 나타내지만 새로운 데이터에 대해서는 낮은 정확도가 나타남
  - ➢ 학습을 통한 궁극적인 목표: 일반화(generalization) 능력 향상

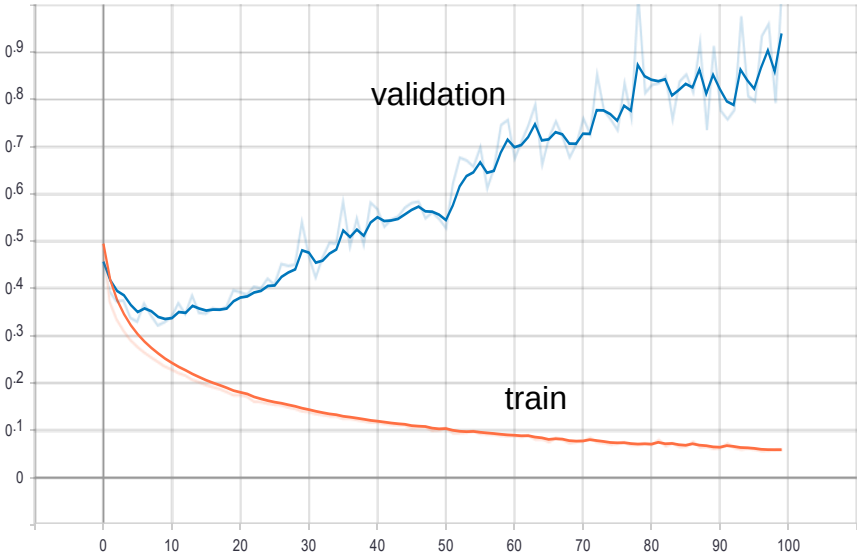


# 과적합

- 과대적합




accuracy




loss

# 좋은 모델이란?

- 현재 데이터(training data)를 잘 설명하는 모델
  - Explanatory modeling
  - Training error를 최소화하는 모델: $MSE_{(training)} = \left(Y - \hat{Y}\right)^2$
- 미래 데이터(testing data)에 대한 예측 성능이 좋은 모델
  - Predictive modeling
  - Expected MSE: $E\left[\left(Y - \hat{Y}\right)^2 | X\right]$ 최소화

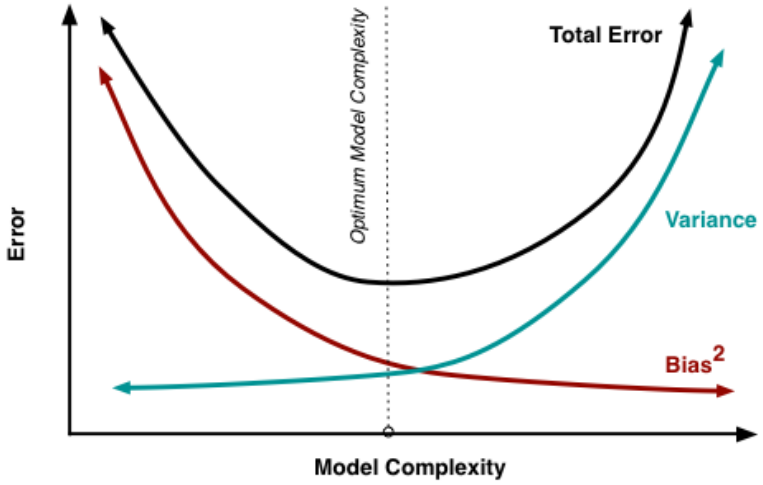
# 좋은 모델이란?

$$E\left[\left(Y-\hat{Y}\right)^2 | X\right]$$
$$= \sigma^2 + \left(E\left[\hat{Y}\right]-\hat{Y}\right)^2 + E\left[\hat{Y}-E\left[\hat{Y}\right]\right]^2$$
$$= \sigma^2 + Bias^2\left(\hat{Y}\right) + Var\left(\hat{Y}\right)$$

Irreducible Error + Bias$^2$ + variance

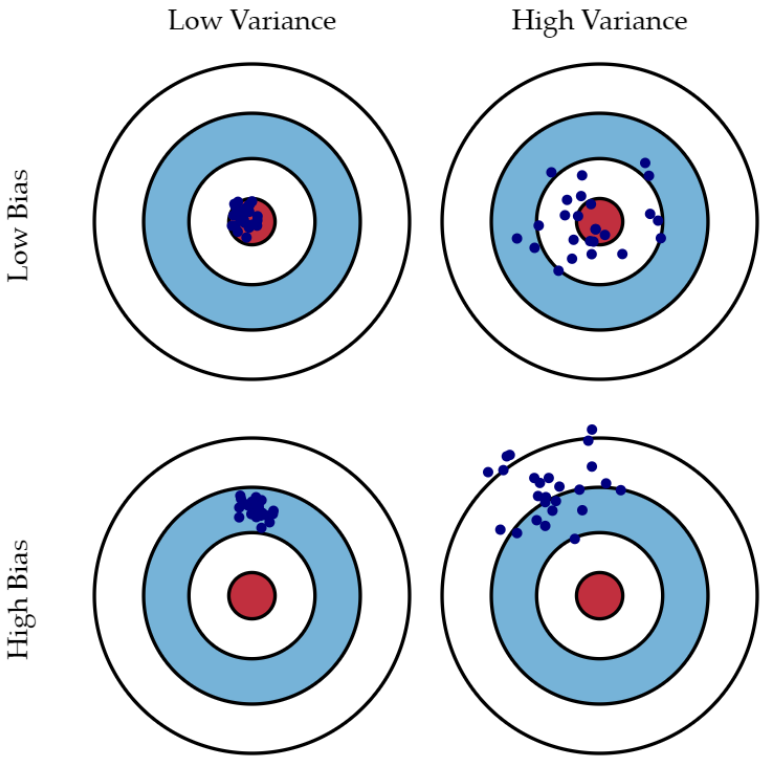
# Bias와 Variance

- 바이어스와 분산은 trade-off 관계
- 바이어스 희생을 최소로 유지하며 분산을 최대로 낮추는 전략 필요



# Bias와 Variance

- Bias
  - 예측 값이 실제 값에서 얼마나 떨어져 있느냐
  - 값이 클수록 큰 편견을 가지고 있음 → underfitting
- Variance
  - 예측 값이 서로 얼마나 흩어져 있느냐
  - 값이 클수록 새로운 입력에 대해 정확하게 예측하지 못함 → overfitting



# 초음파 광물 데이터

- dataset/sonar.csv ~ 광석과 돌 구분
- 1988년, 존스홉킨스대학교의 세즈노프스키(Sejnowski) 교수

# 초음파 광물 데이터

| RangeIndex: 208 entries, 0 to 207 | | | | |
|---|---|---|---|---|
| Data columns (total 61 columns): | | | | |
| # | Column | Non-Null | Count | Dtype |
| 0 | 0 | 208 | non-null | float64 |
| 1 | 1 | 208 | non-null | float64 |
| 2 | 2 | 208 | non-null | float64 |
| 3 | 3 | 208 | non-null | float64 |
| . . . | | | | |
| 55 | 55 | 208 | non-null | float64 |
| 56 | 56 | 208 | non-null | float64 |
| 57 | 57 | 208 | non-null | float64 |
| 58 | 58 | 208 | non-null | float64 |
| 59 | 59 | 208 | non-null | float64 |
| 60 | 60 | 208 | non-null | object |
| dtypes: float64(60), object(1) | | | | |
| memory usage: 99.2+ KB | | | | |

# 초음파 광물 데이터

Rock, Mineral

| | 0 | 1 | 2 | 3 | 4 | ... | 56 | 57 | 58 | 59 | 60 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.02 | 0.0371 | 0.0428 | 0.0207 | 0.0954 | ... | 0.018 | 0.0084 | 0.009 | 0.0032 | R |
| 1 | 0.0453 | 0.0523 | 0.0843 | 0.0689 | 0.1183 | ... | 0.014 | 0.0049 | 0.0052 | 0.0044 | R |
| 2 | 0.0262 | 0.0582 | 0.1099 | 0.1083 | 0.0974 | ... | 0.0316 | 0.0164 | 0.0095 | 0.0078 | R |
| 3 | 0.01 | 0.0171 | 0.0623 | 0.0205 | 0.0205 | ... | 0.005 | 0.0044 | 0.004 | 0.0117 | R |
| 4 | 0.0762 | 0.0666 | 0.0481 | 0.0394 | 0.059 | ... | 0.0072 | 0.0048 | 0.0107 | 0.0094 | R |

| | 0 | 1 | 2 | 3 | 4 | ... | 56 | 57 | 58 | 59 | 60 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 203 | 0.0187 | 0.0346 | 0.0168 | 0.0177 | 0.0393 | ... | 0.0065 | 0.0115 | 0.0193 | 0.0157 | M |
| 204 | 0.0323 | 0.0101 | 0.0298 | 0.0564 | 0.076 | ... | 0.0034 | 0.0032 | 0.0062 | 0.0067 | M |
| 205 | 0.0522 | 0.0437 | 0.018 | 0.0292 | 0.0351 | ... | 0.014 | 0.0138 | 0.0077 | 0.0031 | M |
| 206 | 0.0303 | 0.0353 | 0.049 | 0.0608 | 0.0167 | ... | 0.0034 | 0.0079 | 0.0036 | 0.0048 | M |
| 207 | 0.026 | 0.0363 | 0.0136 | 0.0272 | 0.0214 | ... | 0.004 | 0.0036 | 0.0061 | 0.0115 | M |

# 초음파 광물 데이터

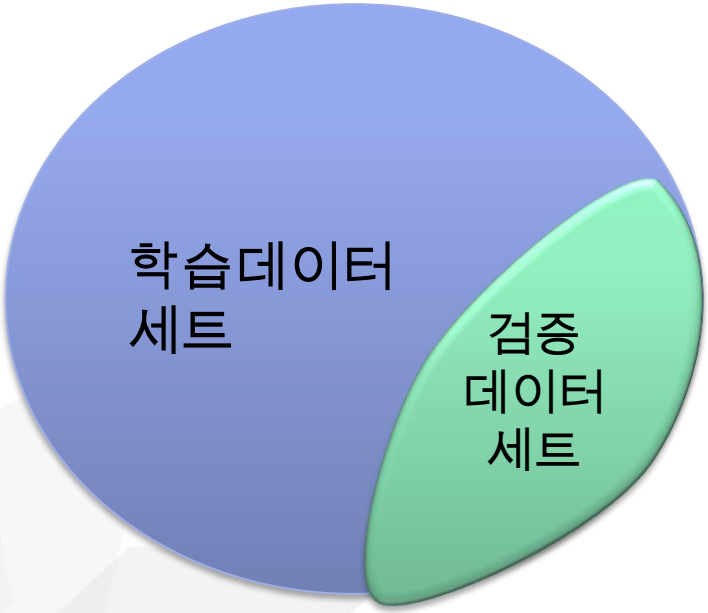
R. Paul Gorman and Terrence J. Sejnowski
Analysis of hidden units in a layered network trained to classify sonar targets
Neural Networks, Volume 1, Issue 1, 1988, Pages 75-89

**TABLE 2**
**Aspect-Angle Dependent Series**

| Number of Hidden Units | Average Performance on Training Sets (%) | Standard Deviation on Training Sets (%) | Average Performance on Testing Sets (%) | Standard Deviation on Testing Sets (%) |
|---|---|---|---|---|
| 0 | 79.3 | 3.4 | 73.1 | 4.8 |
| 2 | 96.2 | 2.2 | 85.7 | 6.3 |
| 3 | 98.1 | 1.5 | 87.6 | 3.0 |
| 6 | 99.4 | 0.9 | 89.3 | 2.4 |
| 12 | 99.8 | 0.6 | 90.4 | 1.8 |
| 24 | 100.0 | 0.0 | 89.2 | 1.4 |

이는 모델 복잡도가 증가할수록 과적합이 발생할 수 있음을 실험적으로 입증한다.

# 데이터세트 유형 구분



# 학습 및 테스트 데이터 구분

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3,
random_state=seed)
```

- Rule of thumb for splitting data
  - In traditional ML (the number of samples ≒ 100 or 1,000 or 10,000) :
  - training/test = 70/30 % OR training/dev/test = 60/20/20 %
  - In large-scale ML such as deep learning (the number of samples 1,000,000) :
  - training/dev/test = 99/1/1 %

# 모델 저장 및 재사용

```
#모델 저장
model.fit(X_train, Y_train, epochs=130, batch_size=5)
model.save('my_model.h5')
```

```
#모델 불러오기
model = load_model('my_model.h5')
model.evaluate(X_test, Y_test)
```

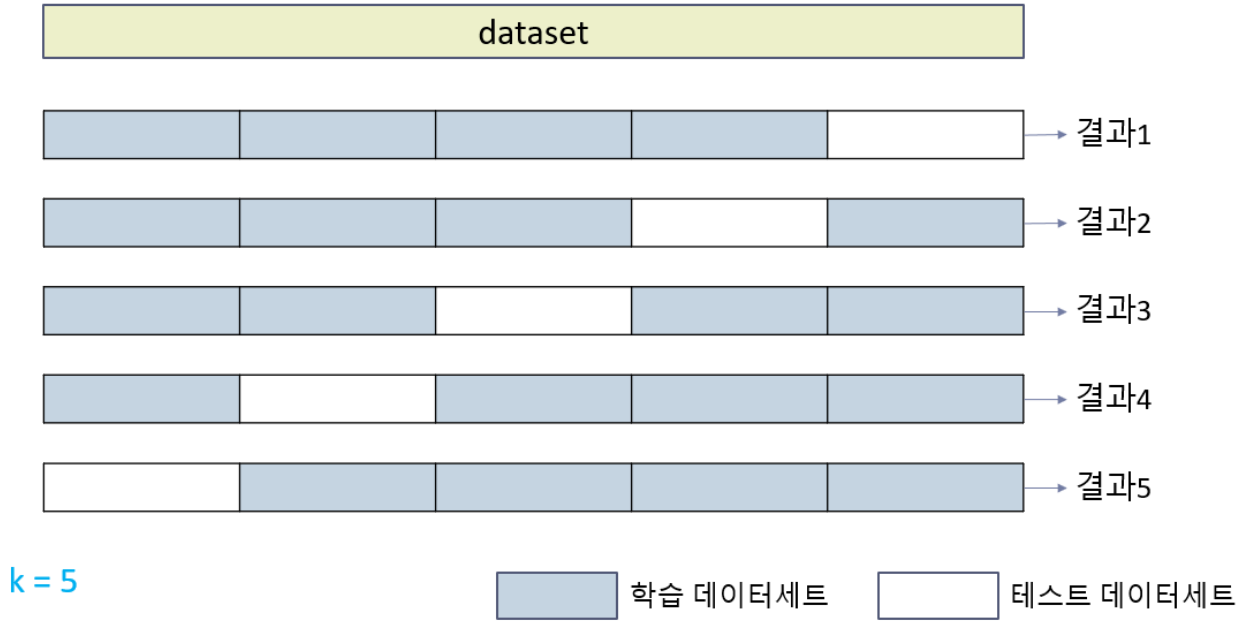
091_01_Sonar.ipynb
sonar.csv

# K-Fold Cross Validation

- 모델 복잡도 ↑ → 과적합 발생
- 훈련 성능만 보고 모델 선택하면 망함
  - 검증 데이터가 필요함
  - 그런데 데이터를 어떻게 나눌 것인가?
  - K-Fold Cross Validation 등장
- 단순 데이터 분할은 부족한가? (데이터가 적은 경우)
  - Train/Test 분할이 운에 좌우
  - 특정 분할에 따라 성능이 크게 달라짐
  - 테이블에서도 표준편차가 존재함 → 분할 민감성 증거

# K-Fold Cross Validation

- 이 테스트 성능이 진짜 일반화 성능인가? : K-Fold Cross Validation 사용
  - ➢ 데이터를 K개로 나누고, 각 조각을 한 번씩 검증 데이터로 사용하여 성능을 평균내는 방법



# K-Fold Cross Validation

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=n_fold, shuffle=True,
random_state=seed)
```

# K-Fold Cross Validation

091_02_Sonar_KFold.ipynb

```
model = Sequential()
model.add(Dense(24, input_dim=60, activation='relu'))
model.add(Dense(10, activation='relu'))
model.add(Dense(1, activation='sigmoid’))

model.compile(loss='mean_squared_error',
              optimizer='adam',
              metrics=['accuracy’])

for train, test in skf.split(X, Y):
    model.fit(X[train], Y[train], epochs=100, batch_size=5)
```

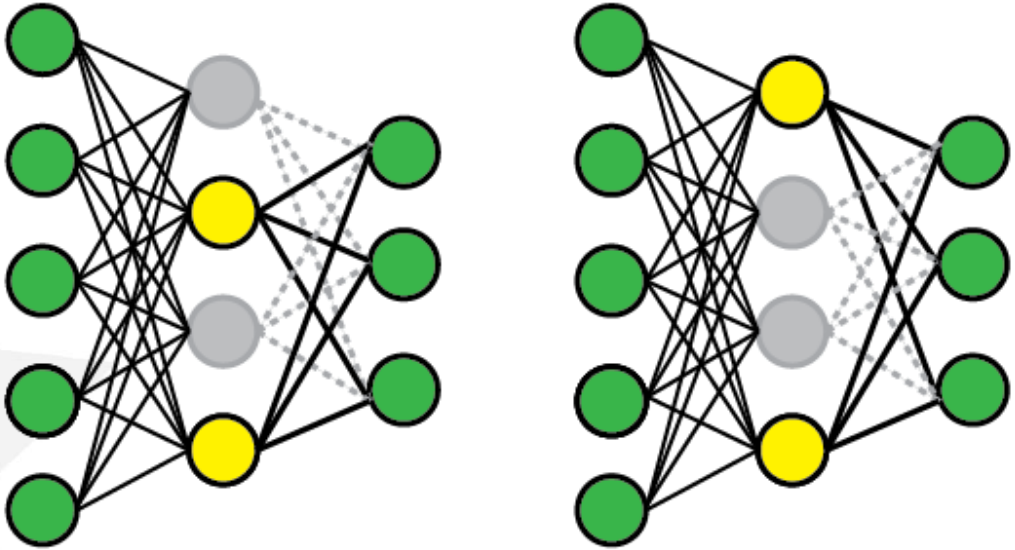
# 과적합 피하기

- Model 기반
  - 규제 (Regularization) : 드랍 아웃 (DropOut)
  - 규제 (Regularization) : 가중치 규제 (Weights Regularization)
  - 배치 정규화 (Batch Normalization)

- Data 기반
  - 데이터 증식 (Data Augmentation)
  - 조기 학습 중단 (Early Stopping)
  - 노이즈 추가 (Noise Injection)

# 과적합 피하기

- 규제 : 드랍 아웃(DropOut)
  - ➢ 학습 과정에서 각 뉴런을 일정 확률로 무작위 제거하여 학습시키는 기법
  - ➢ 매 학습 단계마다 서로 다른 서브 네트워크가 구성되며, 테스트 시에는 전체 네트워크를 사용



# 과적합 피하기

$$\widetilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

- 규제 : 가중치 규제 (Weights Regularization)
  - ➢ 손실함수에 가중치 크기에 대한 페널티 항을 추가하여 모델이 불필요하게 복잡해지는 것을 제한하는 기법

$$H(x) = wx + b$$

Ridge $$J(w,b) = \frac{1}{n}\sum_{k=1}^{n}\left(H\left(x^{(k)}\right) - y^{(k)}\right)^2 + \lambda\boxed{\sum_{k=1}^{n} w_k^2}$$

Lasso $$J(w,b) = \frac{1}{n}\sum_{k=1}^{n}\left(H\left(x^{(k)}\right) - y^{(k)}\right)^2 + \lambda\boxed{\sum_{k=1}^{n} |w_k|}$$

# 과적합 피하기

$$w \rightarrow w - \boxed{\frac{\eta\lambda}{n} sgn(w)} - \eta \frac{\partial C0}{\partial w}$$

$$sgn(w) = \begin{cases} 1, & w > 0 \\ 0, & w = 0 \\ -1, & w < 0 \end{cases}$$

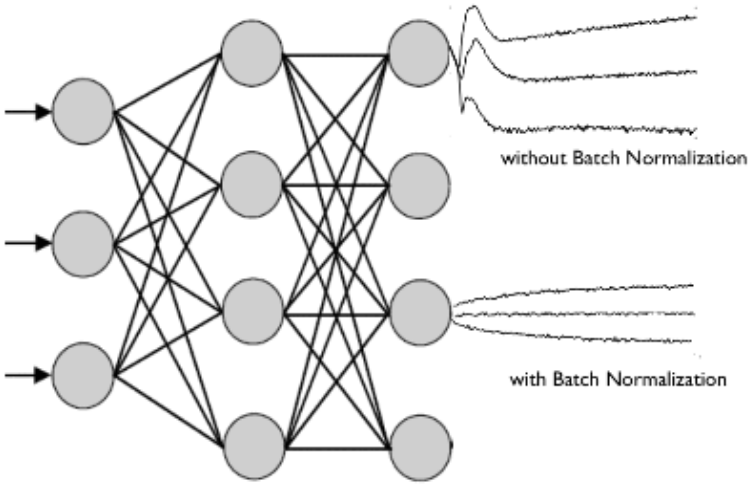
- 규제 : 가중치 규제 (Weights Regularization)
  - ➢ Lasso
    - ✓ L1 norm을 사용한 규제는 가중치의 절댓값을 추가 비용으로 고려하여, 가중치 값을 줄이고 sparse한 모델을 만드는 경향이 있음
    - ✓ L1 규제는 중요하지 않은 특성들의 가중치를 0으로 만들어 feature selection 효과를 가질 수 있기에 Feature Selection에 유리
  - ➢ Ridge
    - ✓ Weight의 크기에 따라 Weight 값이 큰 값을 더 빠르게 감소

$$w \rightarrow w - \eta \frac{\partial C0}{\partial w} - \frac{\eta\lambda}{n}$$

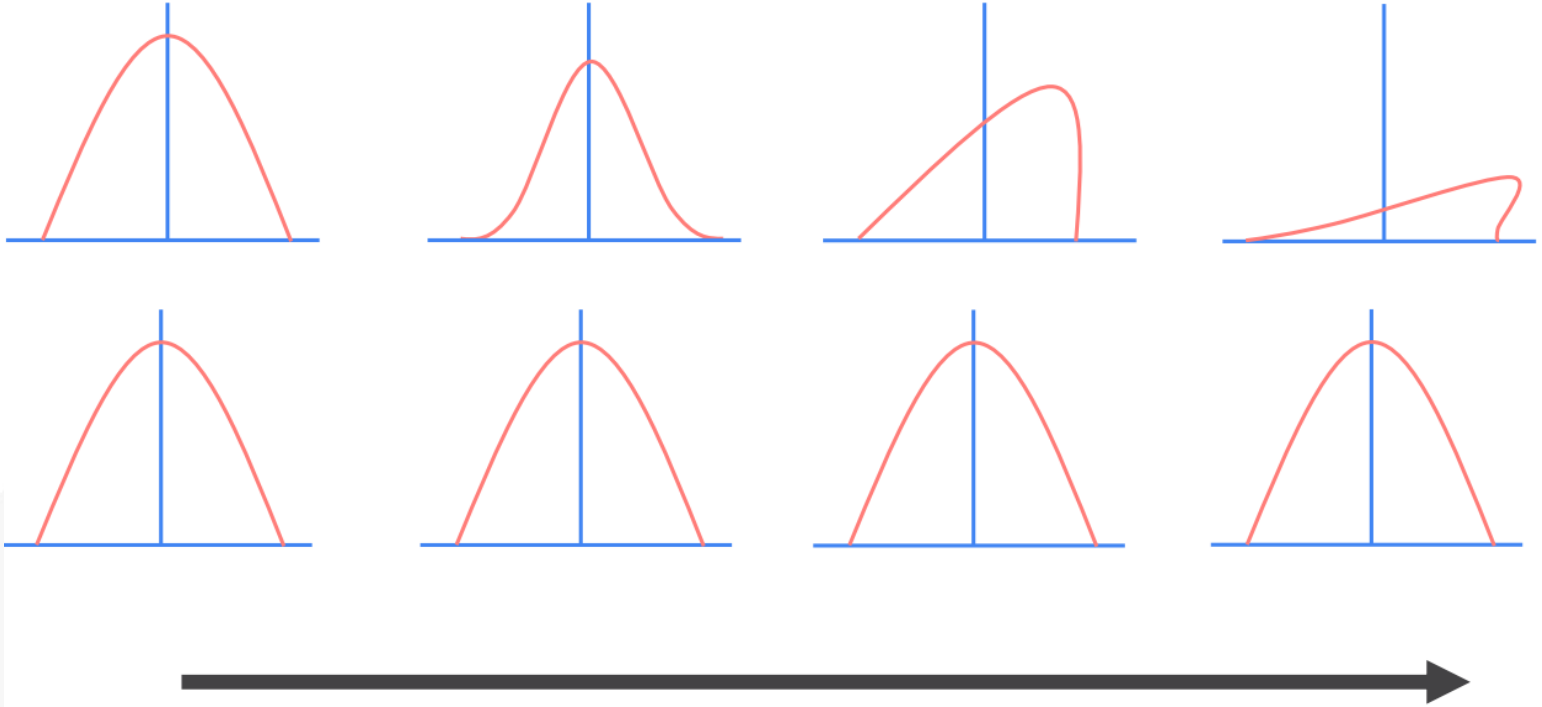
$$= \boxed{\left(1 - \frac{\eta\lambda}{n}\right) w} - \eta \frac{\partial C_0}{\partial w}$$

# 과적합 피하기

- 배치 정규화(Batch Normalization)
  - ➢ Covariate Shift
    - ✓ 이전 레이어의 파라미터 변화로 인하여 현재 레이어의 입력의 분포가 바뀌는 현상
  - ➢ Internal Covariate Shift
    - ✓ 레이어를 통과할 때 마다 Covariate Shift 가 일어나면서 입력의 분포가 약간씩 변하는 현상
  - ➢ 배치 정규화
    - ✓ 미니배치의 평균과 분산을 이용해서 정규화
    - ✓ 학습을 통해 scale (γ) 및 shift (β) 값을 계산
    - ✓ 추론의 경우 학습데이터 전체의 평균과 분산을 사용 (running 평균과 분산 사용)



# 과적합 피하기

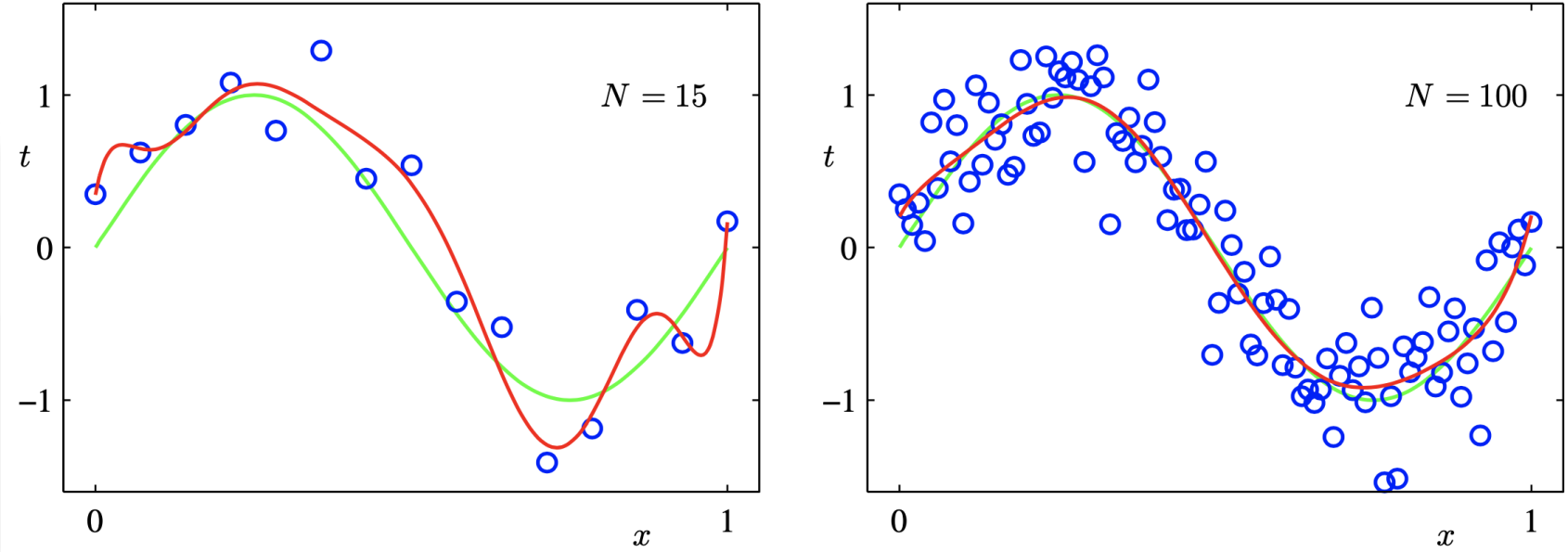


$$\bar{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\hat{x} = \gamma \bar{x} + \beta$$

# 과적합 피하기

- 데이터 증강(Data Augmentation)




(Information science and statistics) Christopher M. Bishop - Pattern Recognition and Machine Learning-Springer (2006)

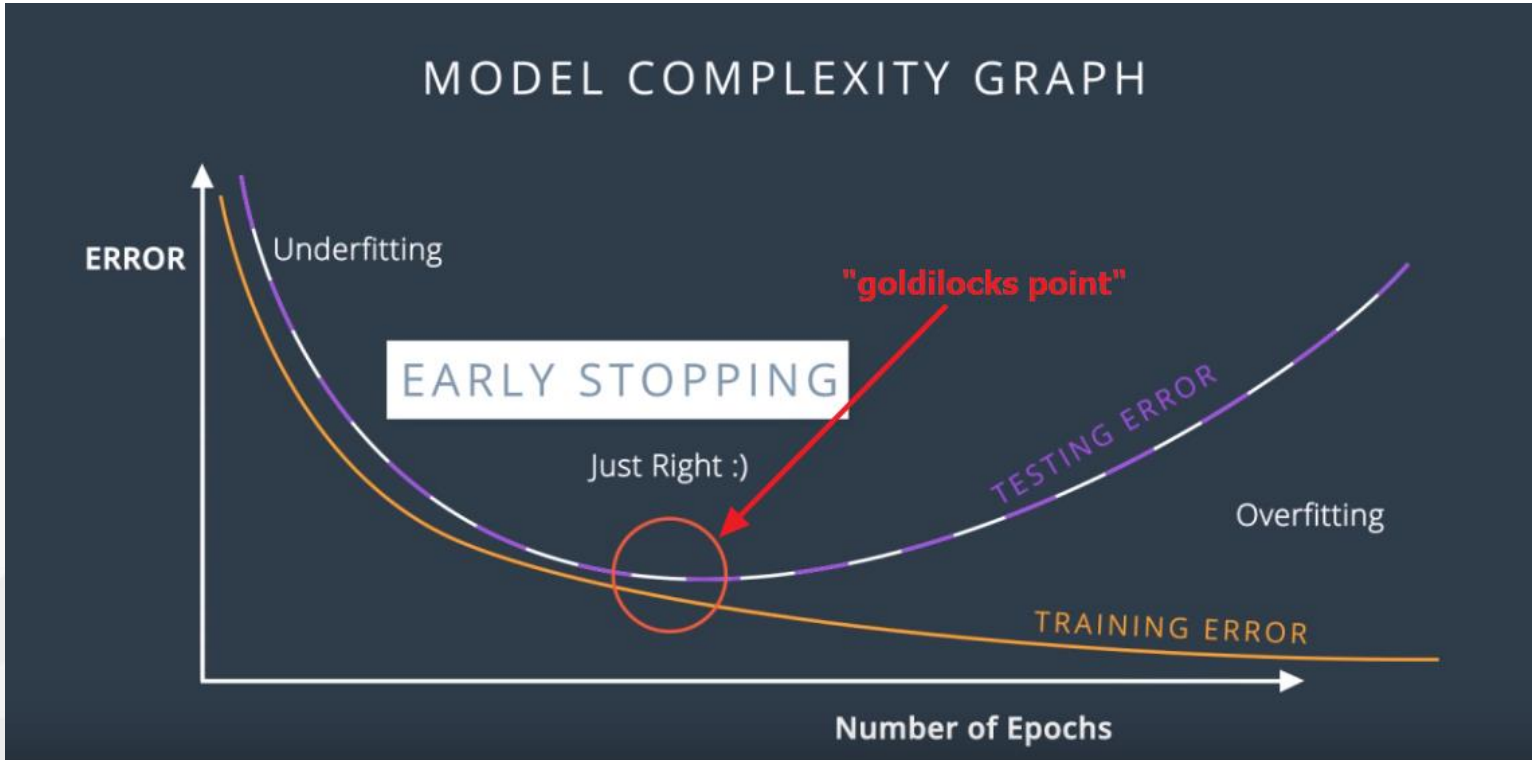
# 과적합 피하기

- 데이터 증강(Data Augmentation)
  - ➢ 기존 데이터를 변형하여 새로운 학습 샘플을 생성하는 기법
  - ➢ 데이터의 의미는 유지하면서 다양성만 증가
  - ➢ 예시
    - ✓ 이미지: 회전, 이동, 반전, 크기 변경
    - ✓ 음성: 노이즈 추가, 속도 변화
    - ✓ 텍스트: 문장 재구성, 동의어 치환

# 과적합 피하기

- 조기 종료(Early Stopping)

검증 데이터 성능이 더 이상 향상되지 않거나 오히려 감소하기 시작하면 **학습을 중단**하는 방법이다.



# 실습

- 모델의 용량이 작아 오차가 큰 현상을 무엇이라고 하는가?
- 학습 데이터에서는 성능이 좋지만 새로운 데이터에서 성능이 나쁜 현상을 무엇이라고 하는가?
- 예측 값이 실제 값에서 얼마나 떨어져 있는지를 나타내는 것은 무엇인가?
- 예측 값이 서로 얼마나 흩어져 있는지를 나타내는 것은 무엇인가?
- 학습 과정에서 각 뉴런을 일정 확률로 무작위 제거하는 기법은 무엇인가?
- 검증 데이터 성능이 더 이상 향상되지 않으면 학습을 중단하는 방법은 무엇인가?
- 데이터를 여러 개로 나누어 각 조각을 한 번씩 검증 데이터로 사용하는 방법은 무엇인가?
- 손실 함수에 가중치 크기에 대한 페널티 항을 추가하여 모델이 복잡해지는 것을 제한하는 기법은?
- 기존 데이터를 변형하여 새로운 학습 샘플을 생성하는 기법은 무엇인가?
- 가중치의 절댓값(L1 norm)을 사용하는 규제 방법은 무엇인가?
- 가중치의 제곱(L2 norm)을 사용하는 규제 방법은 무엇인가?

# 참고 문헌

- 배치 정규화
  - https://youtu.be/m61OSJfxL0U?feature=shared
  - https://youtu.be/JJZZWM9tGp4?feature=shared
- Deep Learning by Google
  - https://www.udacity.com/course/deep-learning--ud730
- 기계학습, 오일석 저, 한빛미디어